

Trabalho Prático 3

Disciplina: Arquitetura de Computadores

Instituição: PUC Minas

Alunos: André Vieira Penchel, Arthur de Sá Camargo, Henrique Saldanha Mendes Veloso, João Victor Ferreira Pena, Matheus de Oliveira Campello

Data: 02/07/2026

1. Introdução

A multiplicação de matrizes é uma das operações mais vitais da computação, sendo aplicada em áreas como inteligência artificial e computação gráfica. Dada essa relevância, é seguro dizer que métodos que permitam a conclusão mais rápida dessa operação tem impacto significativo para os mais diversos tipos de aplicações. Com isso em mente, torna-se relevante a discussão de estratégias e algoritmos que possibilitam essa aceleração.

Neste trabalho serão abordadas as estratégias de paralelização (tanto em CPU quanto em GPU), tiling e vetorização SIMD, comparando os ganhos de cada caso variando tamanho de entrada e grau de paralelização.

2. Metodologia Experimental

Ambiente de Hardware

Os experimentos foram conduzidos no ambiente do Google Colab:

CPU: Processador AMD EPYC 7B12Z

GPU: NVIDIA Tesla T4

Procedimento de Teste

O problema estudado é a multiplicação de matrizes densas quadradas (GEMM), definida como $C = A \times B$, com A , B e C de dimensão $N \times N$ e elementos em ponto flutuante de precisão simples. O número de operações de ponto flutuante realizadas por uma multiplicação de matrizes desse tipo é aproximadamente $2N^3$ (uma multiplicação e uma soma por elemento do produto interno, para cada uma das N^2 posições de saída).

A partir do tempo médio de execução t , o desempenho é reportado em GFLOP/s:

$GFLOP/s = 2N^3 / (t \times 10^9)$

Os experimentos foram organizados para avaliar isoladamente o efeito de cada otimização aplicada sobre a implementação de referência (naive), permitindo isolar a contribuição de:

- localidade de memória (uso da matriz B transposta);
- blocagem/tiling (particionamento do produto em sub-blocos que cabem em cache);
- vetorização automática pelo compilador (SIMD);
- paralelismo de memória compartilhada com OpenMP;
- aceleração em GPU com CUDA (kernel ingênuo e kernel com memória compartilhada/tiled).

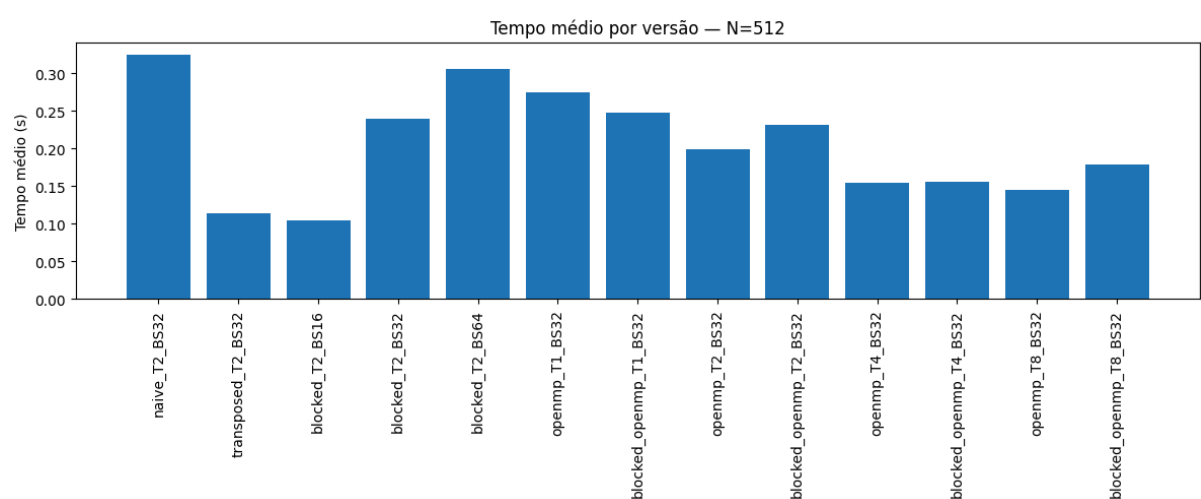
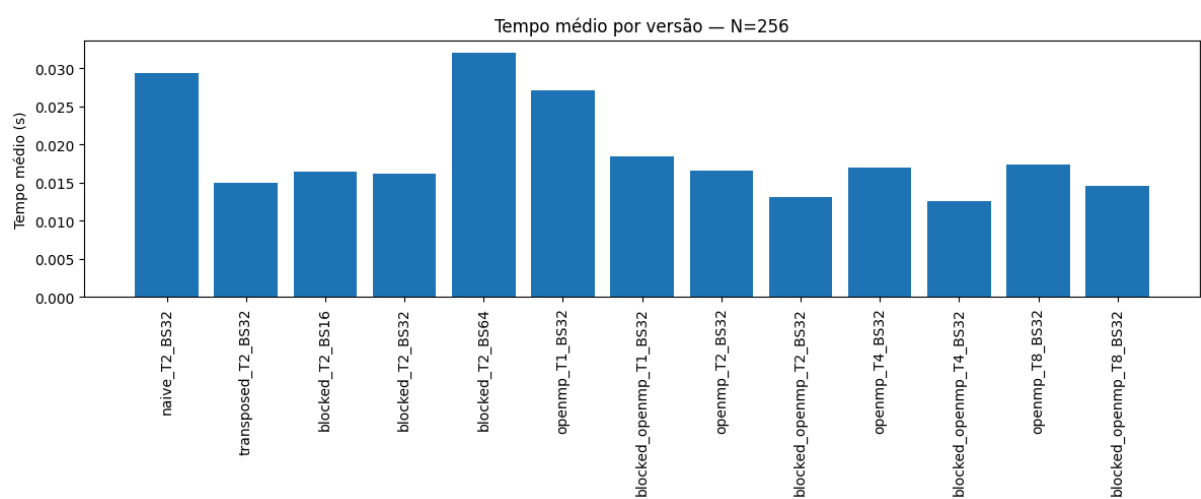
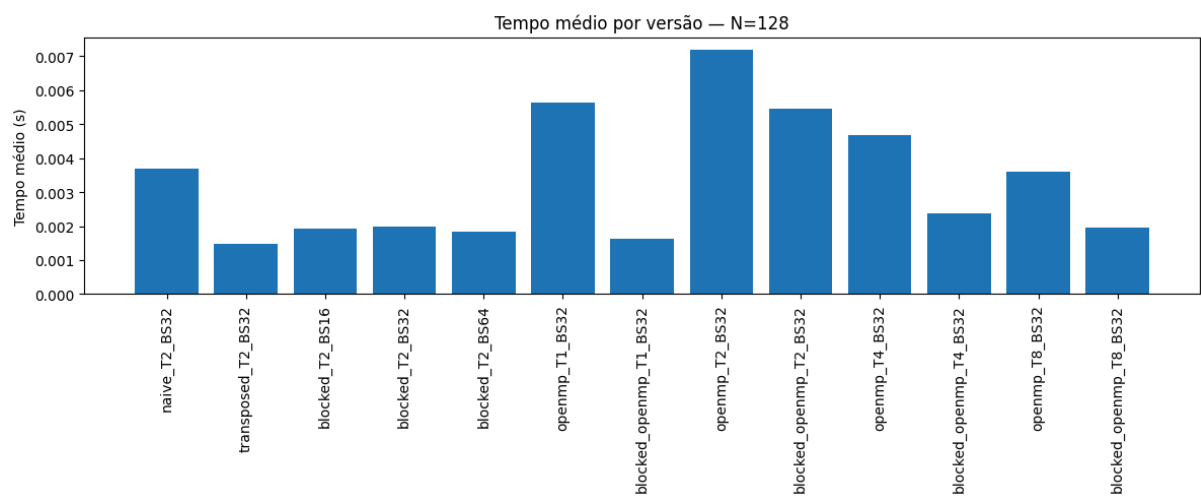
Os testes foram feitos em quintuplicatas visando à redução de possíveis ruídos, utilizando a média aritmética para extrair os resultados finais.

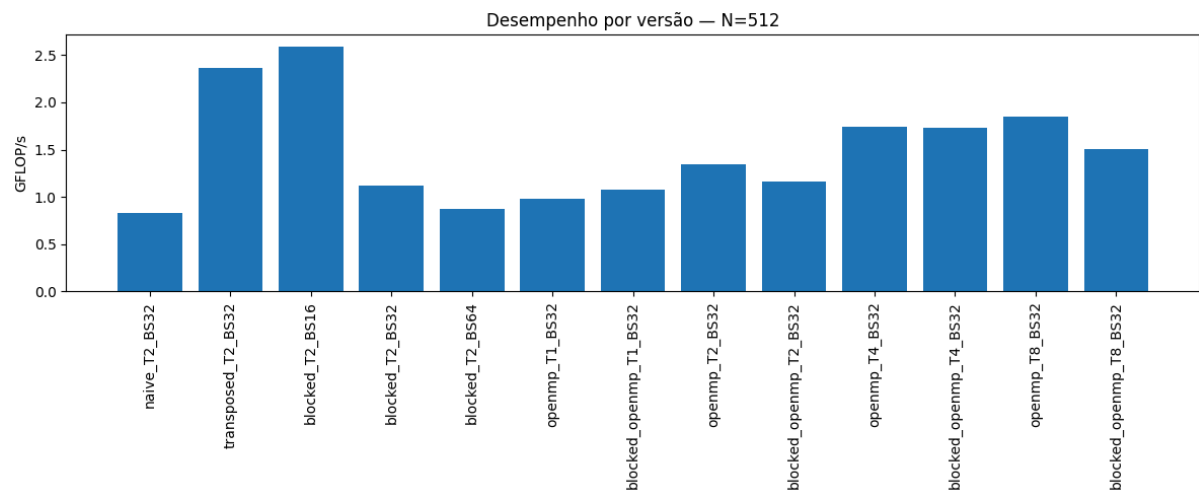
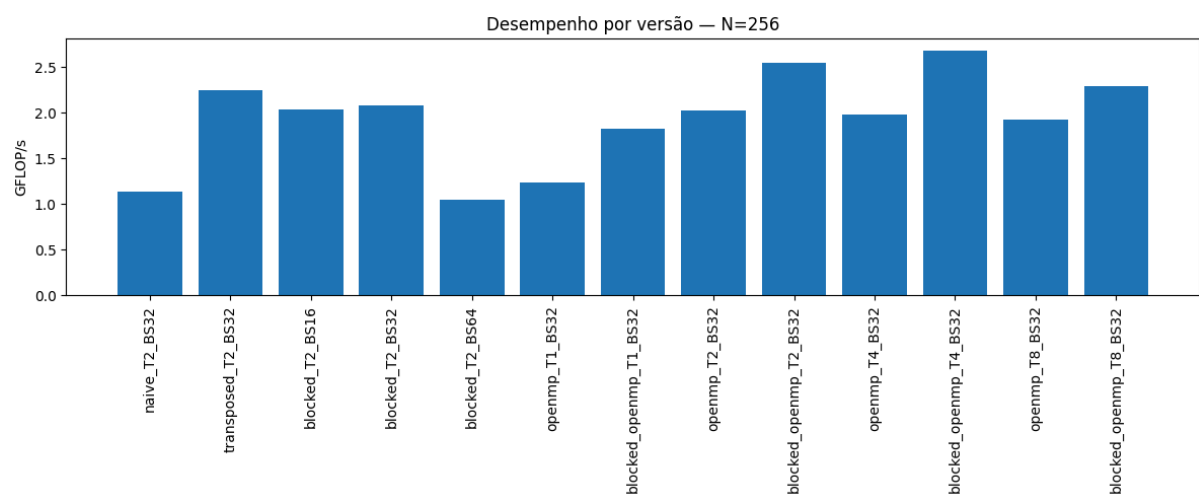
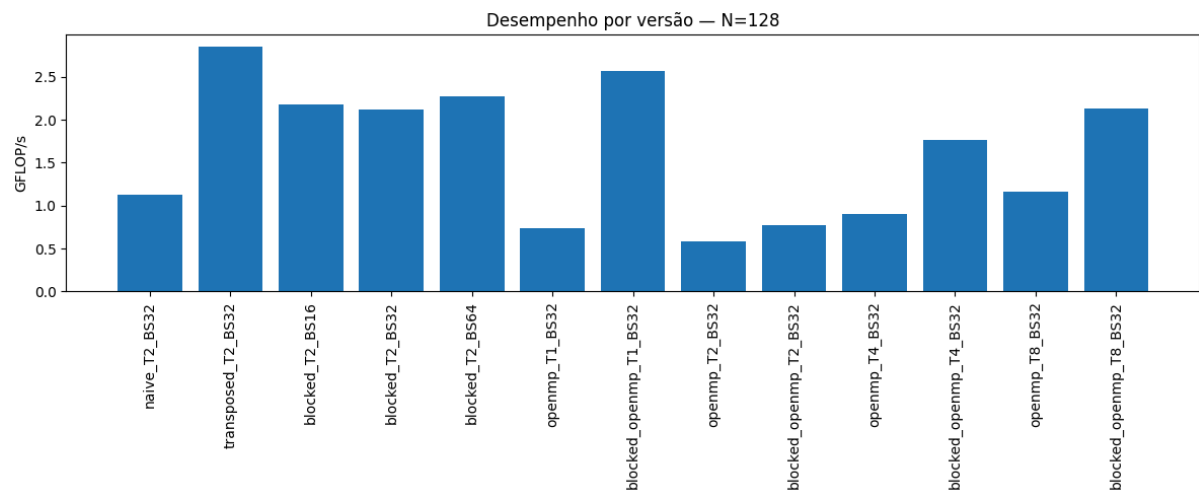
3. Resultados e Análise de Desempenho

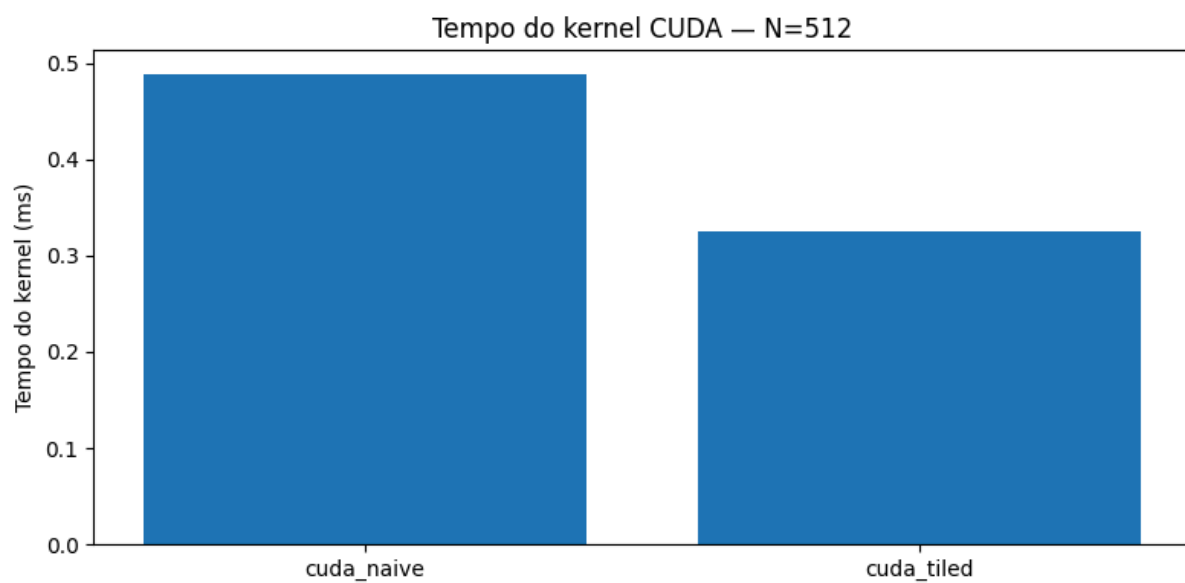
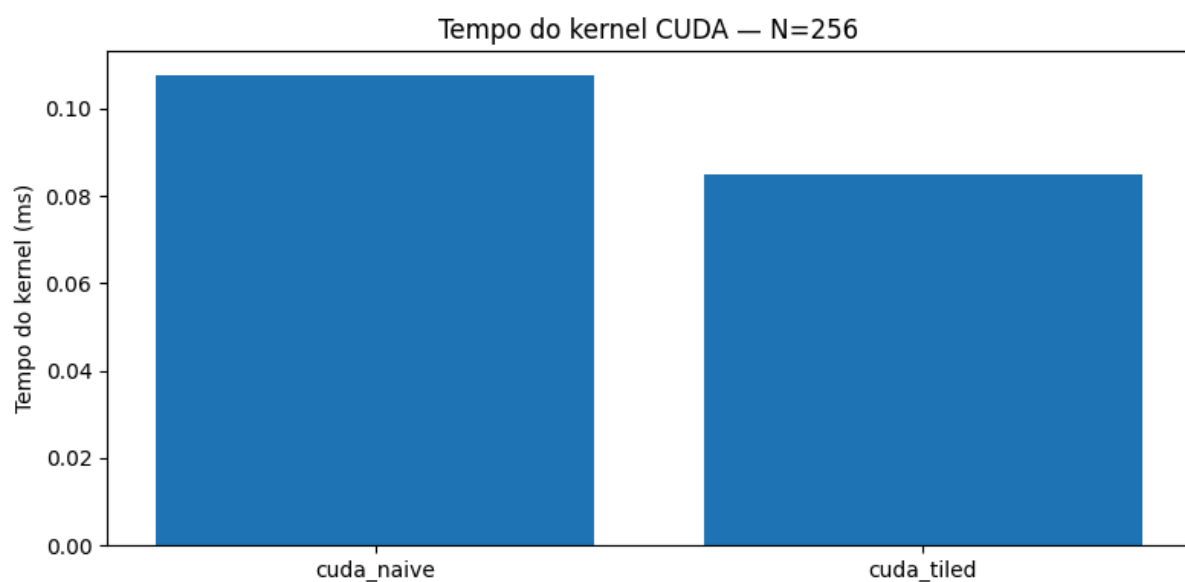
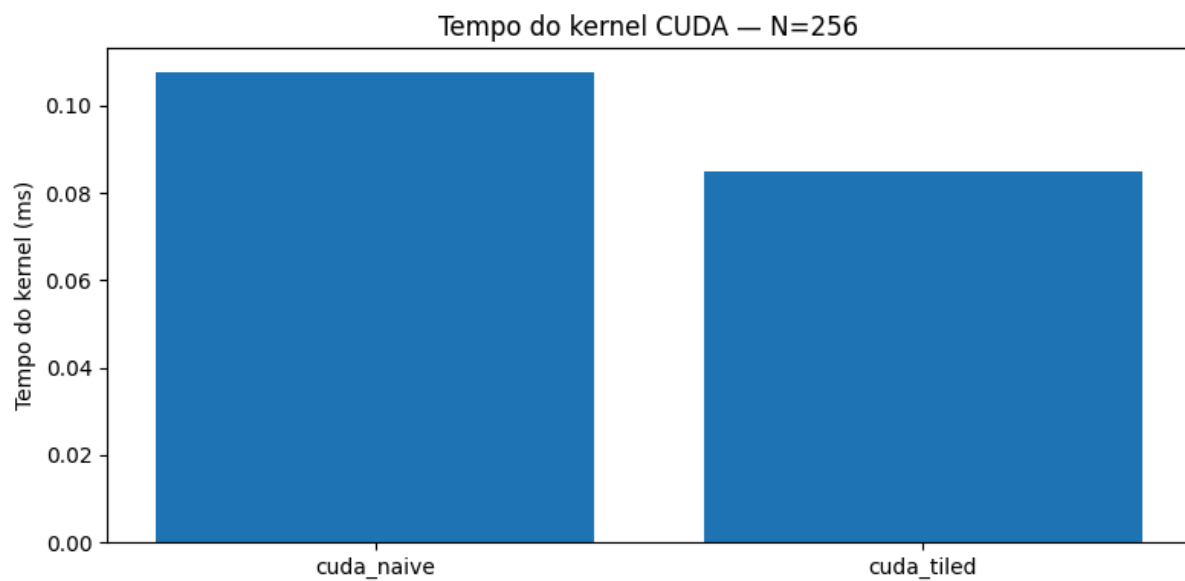
A tabela abaixo resume os resultados obtidos para uma matriz de dimensão 128×128. Vale destacar que os tempos de execução via CUDA incluem o tempo de transferência de dados da RAM para VRAM, e este impacto será discutido posteriormente

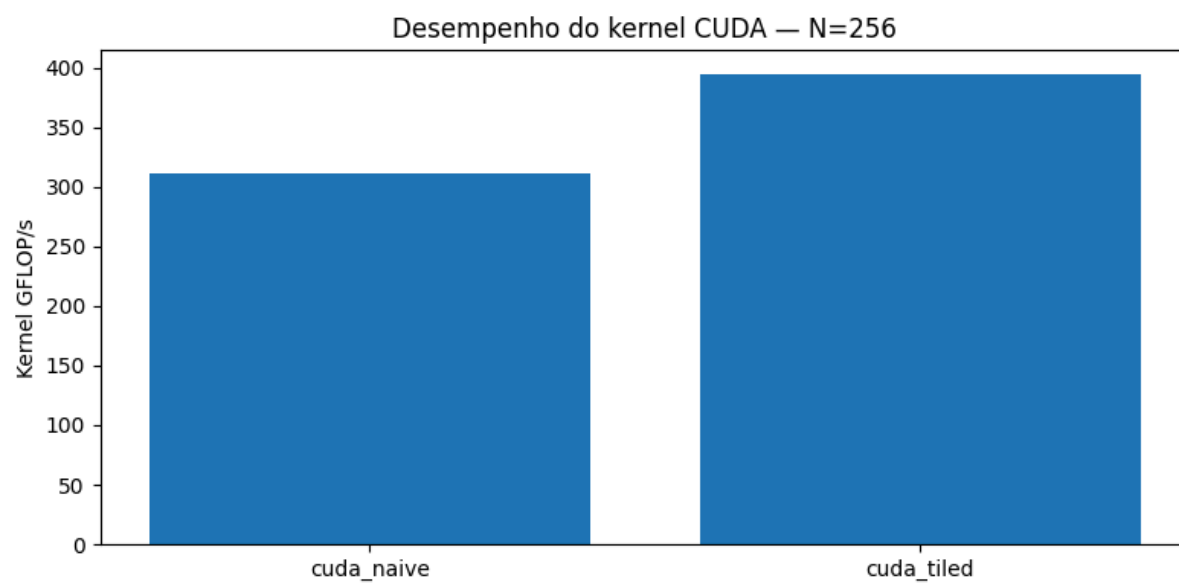
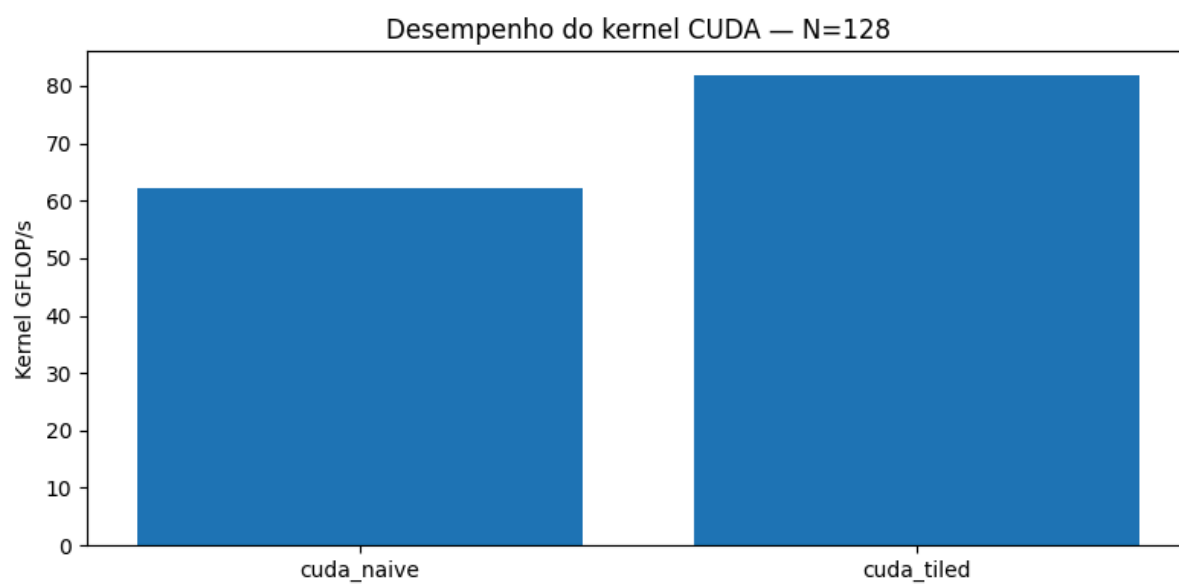
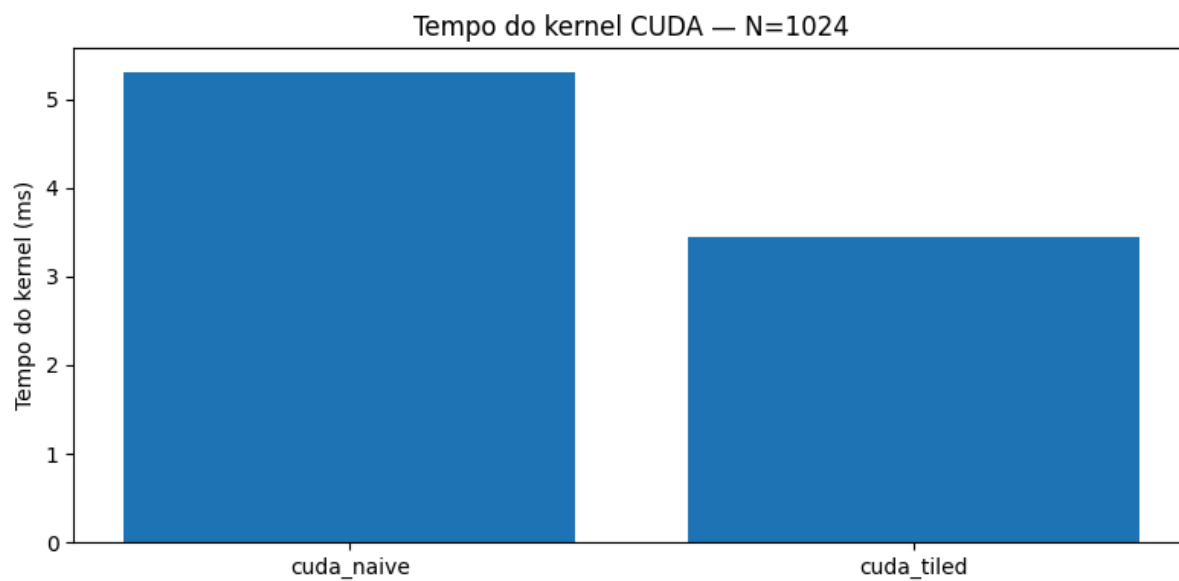
Versão Avaliada	Tempo Médio (s)	GFLOP/s	Speedup	Eficiência
CPU Naive	0.003377	1.242007	1.00x	100%
CPU Transposta	0.002689	1.559872	1.25x	100%
CPU OpenMP 2 threads	0.004173	1.005121	0.8x	40%
CPU OpenMP 4 threads	0.003392	1.236693	0.99x	99%
CUDA Naive	0.177043	23.6908	0.01x	N/A
CUDA Tiled	0.159974	81.9199	0.02x	N/A

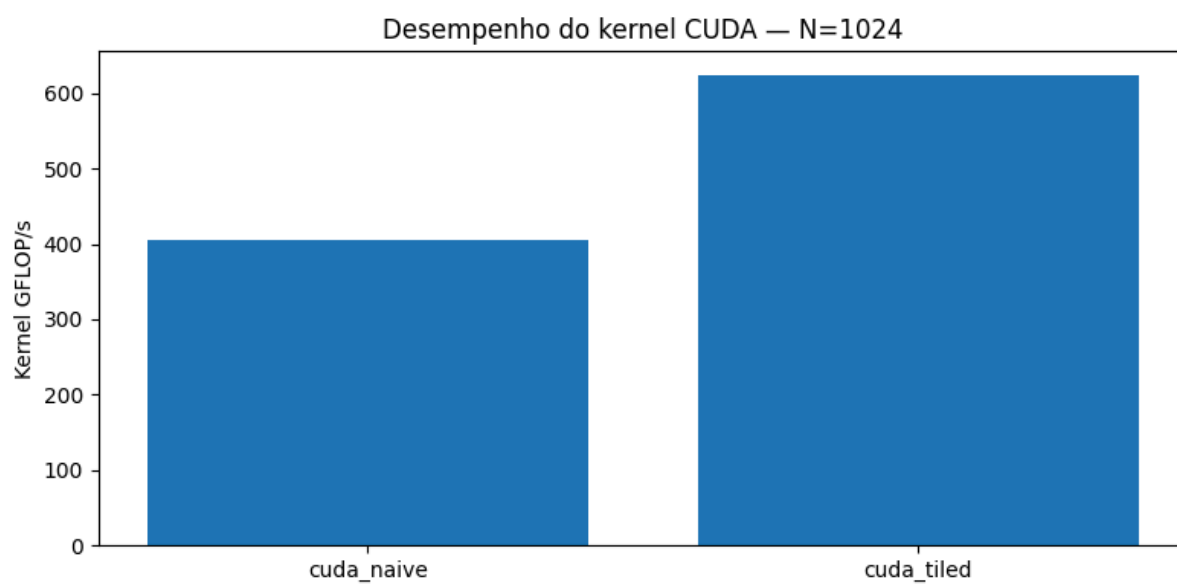
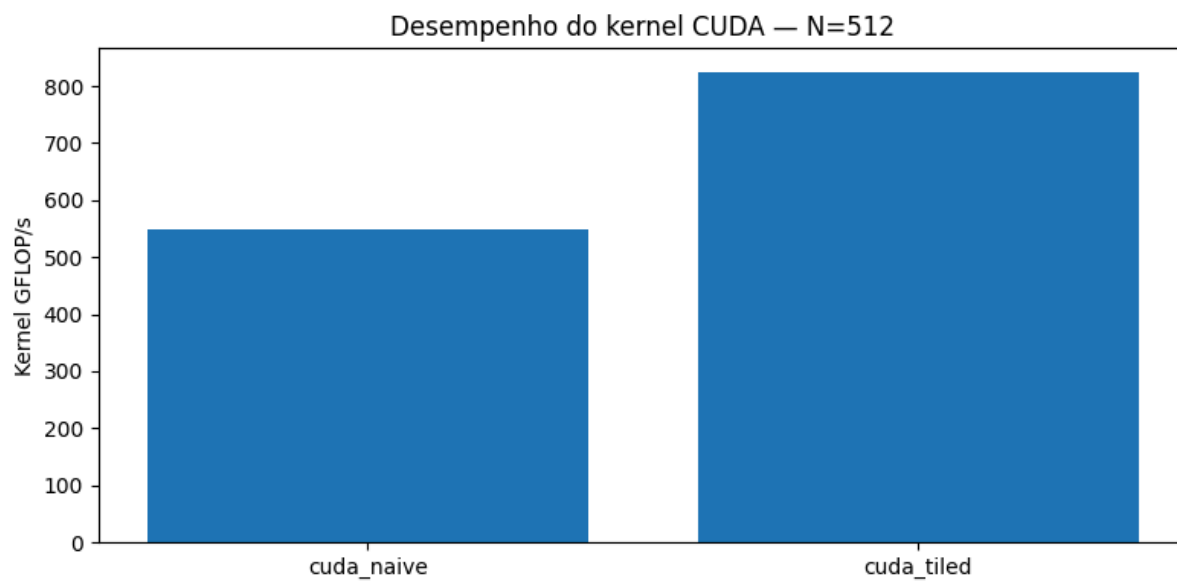
Gráficos gerados

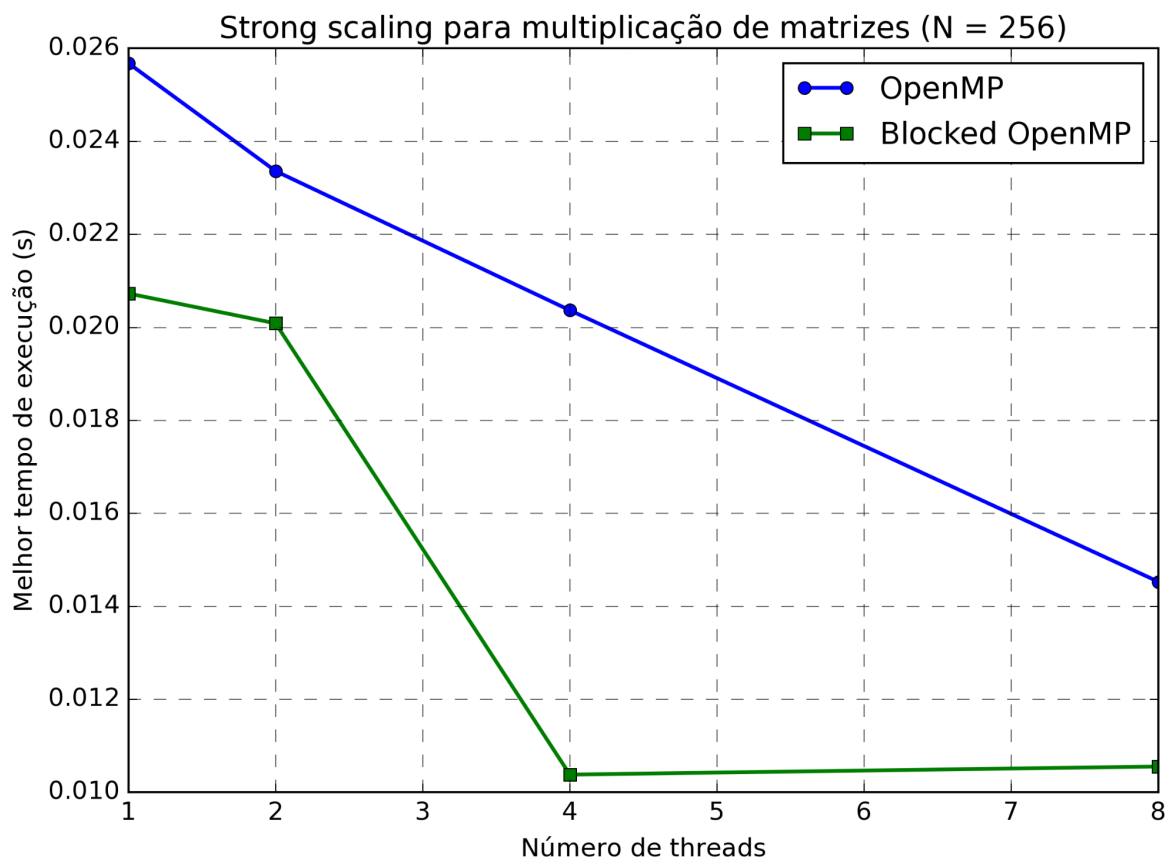
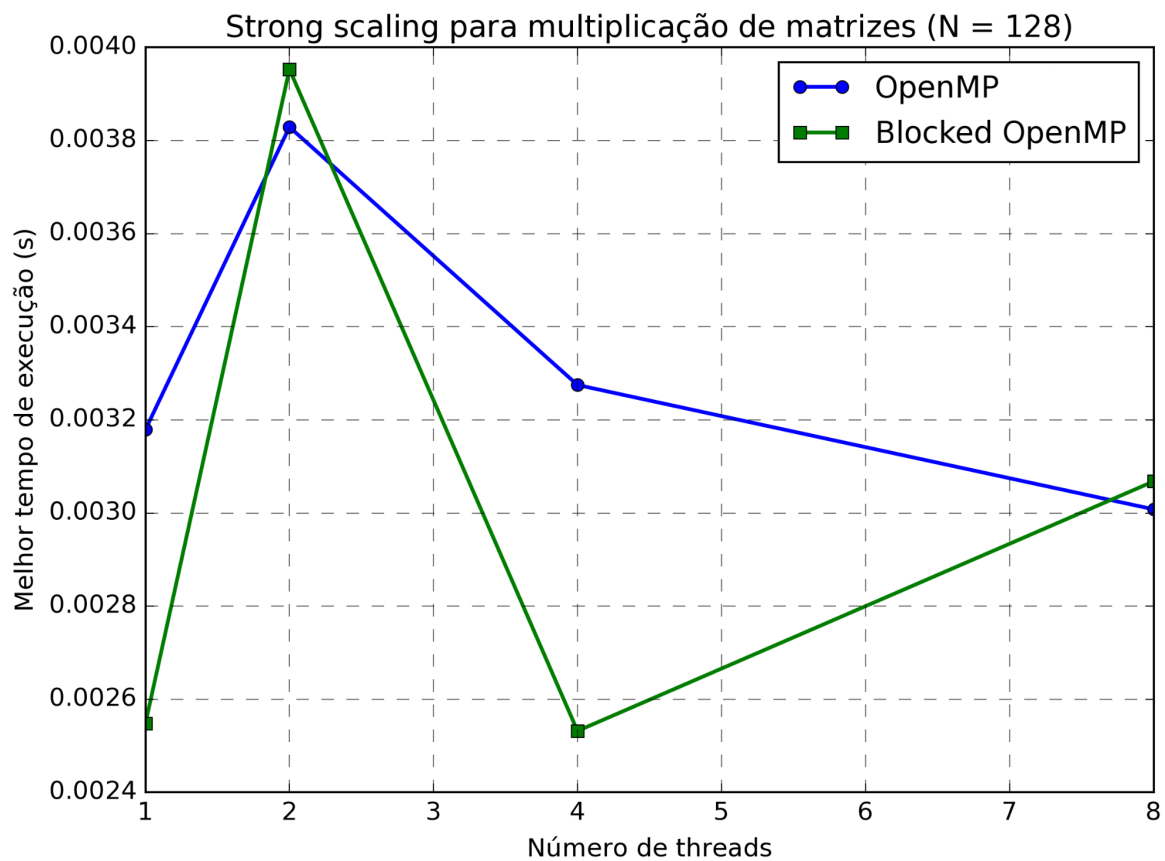


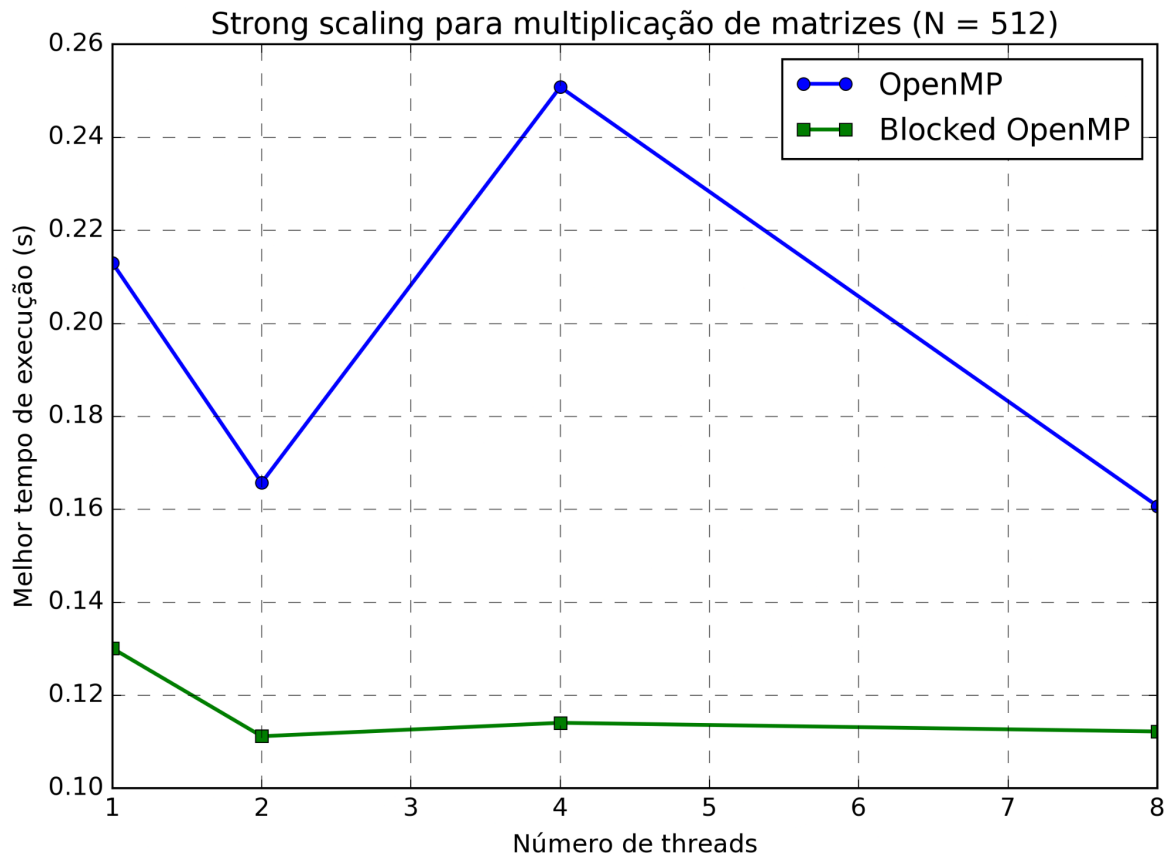












4. Discussão dos Resultados

4.1 Por que a versão ingênua apresenta baixo desempenho?

Considerando duas matrizes, A e B, a versão naive sofre com misses de cache constantes. Ao acessar a matriz B, o loop interno acessa os elementos por coluna. Isso faz com que a cada acesso a CPU precise ir até a RAM buscar os dados ao invés de usar a localidade espacial, aumentando drasticamente o tempo de execução.

4.2 O impacto da transposição de B foi significativo? Por quê?

A transposição da matriz B transformou o acesso de coluna em acesso de linha. O impacto foi significativo pois permitiu que múltiplos elementos carregados em uma única linha de cache fossem processados consecutivamente, reduzindo a latência do acesso de memória e permitindo speedup de 1.25x.

3. Qual tamanho de bloco apresentou melhor desempenho? Por quê?

Os blocos de $BS = 16$ apresentaram o melhor resultado de tempo independente do tamanho da entrada. Esse resultado se deve, provavelmente, porque esse tamanho de bloco cabe inteiramente na cache, diminuindo o overhead de busca de dados na memória principal. Ainda assim, comparado ao método naïve, todos os valores de BS apresentaram aceleração: para os valores de BS iguais à 16, 32 e 64, os speedups foram de 1.37x, 1.33x e 1.1x, respectivamente

4. O compilador conseguiu vetorizar alguma parte do código? Use o relatório de vetorização como evidência.

```
gemm_cpu.c:67:31: optimized: loop vectorized using 32 byte vectors
gemm_cpu.c:67:31: optimized: loop vectorized using 16 byte vectors
gemm_cpu.c:98:9: optimized: basic block part vectorized using 32 byte vectors
gemm_cpu.c:111:9: optimized: basic block part vectorized using 32 byte vectors
gemm_cpu.c:111:9: optimized: basic block part vectorized using 32 byte vectors
gemm_cpu.c:67:31: optimized: loop vectorized using 32 byte vectors
gemm_cpu.c:67:31: optimized: loop vectorized using 16 byte vectors
gemm_cpu.c:111:9: optimized: basic block part vectorized using 32 byte vectors
gemm_cpu.c:111:9: optimized: basic block part vectorized using 32 byte vectors
```

O relatório de vetorização indica o uso de vetorizações de 32 e 16 bytes para loops e blocos de código, como evidenciado nas linhas de código acima.

5. O speedup com OpenMP foi próximo do ideal? Explique.

Não. O speedup ideal seria linear, o que é praticamente inalcançável. Além disso, percebe-se na verdade redução da velocidade quando comparado ao sequencial puro. Além de nem todo o programa ser paralelizável, como estabelecido anteriormente, o principal limitador da execução naïve é a transferência de dados para a cache. As tabelas abaixo consideram $N=128$.

Threads	Speedup real	Ideal
4	0.84x	4x
8	0.99x	8x

Utilizar a estratégia de tiling melhora o resultado da paralelização justamente por aliviar o overhead de acesso à RAM. No caso de 4 threads, a melhora é significativa. No caso de 8 threads, analisando o tempo médio houve novamente uma piora (velocidade 0.8x do naive sequencial), no entanto, para a tabela abaixo, foi considerado o melhor tempo alcançado, pois suspeita-se que a causa do slowdown foi o ambiente de execução do Google Colab.

Threads (Tiled)	Speedup real	Ideal
4	1.33x	4x
8	1.1x	8x

6. A eficiência diminuiu com mais threads? Por quê?

Sim. A eficiência se manteria apenas em 100% caso ambos os núcleos fossem usados em sua total capacidade durante toda a execução, no entanto existem partes do programa que são invariavelmente sequenciais, além do overhead necessário para paralelizar e delegar as execuções. Isso faz com que seja impossível que a eficiência se mantenha em 100%.

7. A GPU foi mais rápida considerando apenas o kernel?

Versão Avaliada	Tempo Médio para N = 128	Tempo Médio para N = 256	Tempo Médio para N = 512
CPU OpenMP 8 Threads	0.00441	0.015038	0.169967
GPU Naive (apenas kernel)	0.067462	0.107738	0.488518

Como é possível observar, mesmo considerando apenas o tempo de execução de kernel, a paralelização via GPU ainda foi significativamente mais lenta. Esse padrão persiste mesmo para a versão que utiliza a estratégia de blocos.

Versão Avaliada	Tempo Médio para N = 128	Tempo Médio para N = 256	Tempo Médio para N = 512
CPU OpenMP 8 Threads Tiled	0.004135	0.011349	0.118699
GPU Tiled (apenas kernel)	0.050259	0.091648	0.332902

8. A GPU foi mais rápida considerando o tempo total com cópias?

Não. Na verdade, a cópia dos dados para a VRAM e da VRAM para a memória principal acrescenta um overhead significativo. Tomando como exemplo a matriz de dimensões 128×128, que possui a menor quantidade de dados para transferência, é possível perceber que não só o tempo de execução se tornou ainda maior, mas 63% de todo o tempo de execução foi utilizado apenas para transferir dados. Aumentando as dimensões da matriz para 1024×1024, percebe-se que o tempo de processamento aumenta quando comparado ao tempo de transferência, alcançando até 49.5%, o que é mais equilibrado, mas o tempo de execução ainda é predominantemente usado para transferência de dados.

Versão Avaliada	Comunicação GPU ↔ CPU	Tempo Total	Transferência/Total
GPU Naive N=128	0.1049	0.16551	63.3%
GPU Naive N=1024	3.3937	6.71619	50.5%

9. Qual foi a principal limitação observada: computação, memória, comunicação ou overhead?

Para a GPU, o overhead foi o principal limitador. Como indicado anteriormente, a cópia dos dados é a maior parte do tempo de execução. Para a CPU, por outro lado,

o principal limitador é o tamanho da cache, e por isso as estratégias de speedup incluem usar a cache estrategicamente.

10. Quais cuidados são necessários ao usar o Google Colab para avaliar desempenho?

As GPUs e CPUs do Colab são compartilhadas com outros usuários, fazendo com que o desempenho possa variar entre execuções mesmo com o mesmo código. Além disso, o Colab pode alocar diferentes modelos de GPU, para tornar os resultados comparáveis. Portanto, o ideal é usar várias execuções para diminuir o ruído.

5. Conclusão

As descobertas deste trabalho demonstram que as otimizações de CPU via transposição e tiling foram essenciais para melhorar o desempenho através do uso eficiente da cache. Por outro lado, o paralelismo com OpenMP enfrentou limitações devido ao overhead e trechos sequenciais, não atingindo ganhos lineares. Por fim, o alto custo de transferência de dados entre memória principal e VRAM revelou que a GPU só se torna vantajosa quando o ganho computacional compensa o overhead de comunicação, o que não foi observado para as dimensões de matrizes testadas nos experimentos realizados.

6. Uso de IA

Inteligência artificial foi utilizada para gerar os scripts de python que geraram os gráficos de strong scaling, para formatar o relatório e para inspiração de pequenas partes do texto. A maior parte do relatório foi feita sem esse recurso.